

DATA 260 – Homework 4 Report

Student ID: 018322837

Name: Shashank Ranjan

Section 0 : Personal Configuration and Domain

Name	Definition	Value
SID	Student ID	018322837
SID4	Last 4 of Student ID	2837
PORT_BASE	$8000 + \text{SID} \bmod 900(137)$	8137
prefix	"s" + SID4	s2837
Seed	SID4	2837
verify sid	$260000 + \text{SID4}$	262837
DOMAIN_ID	SID4 MOD 8	5
Assigned Domain	Assignment domain	Local restaurant inspections

Repository: <https://github.com/ShankySJSU/Data260-2837>

Submission Tag: hw4

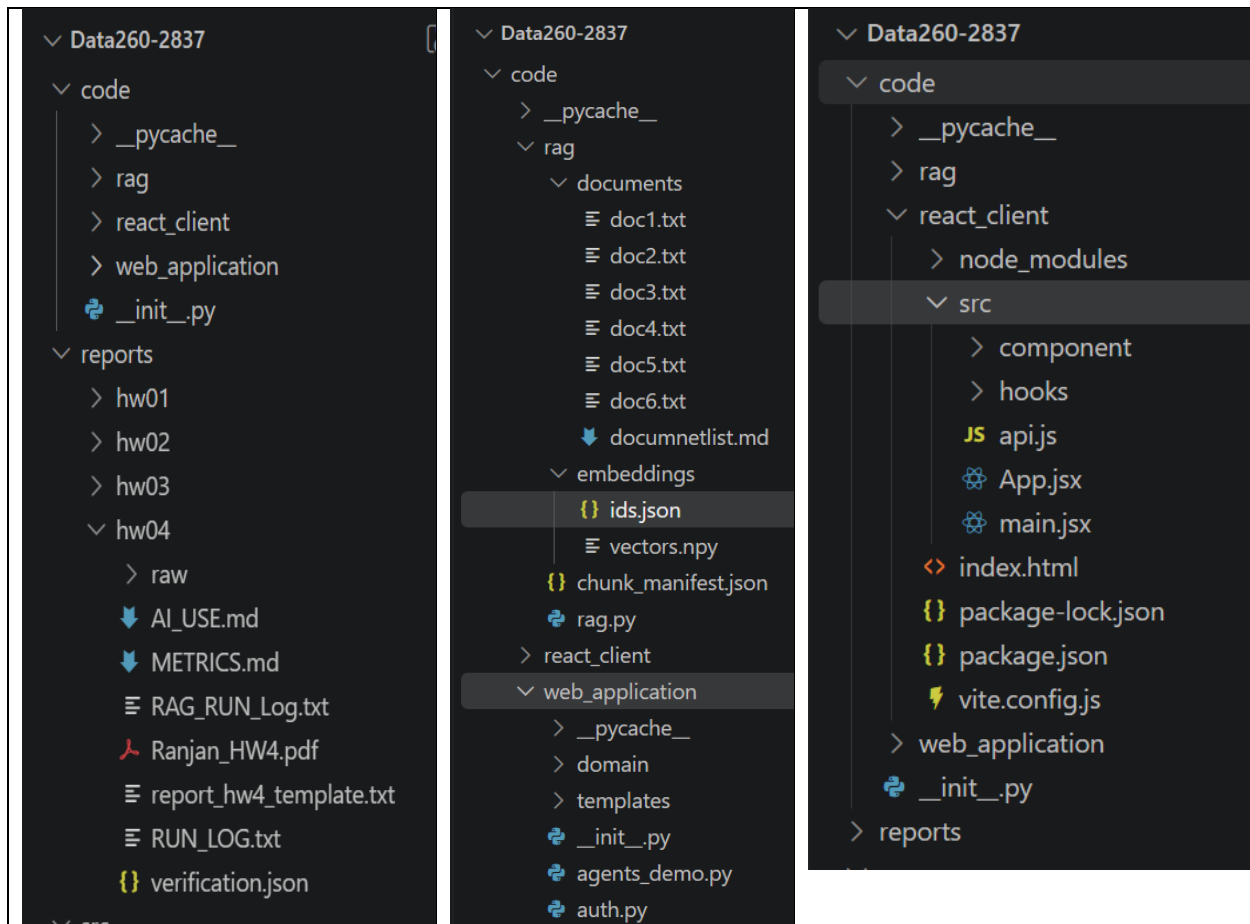
Hardware:

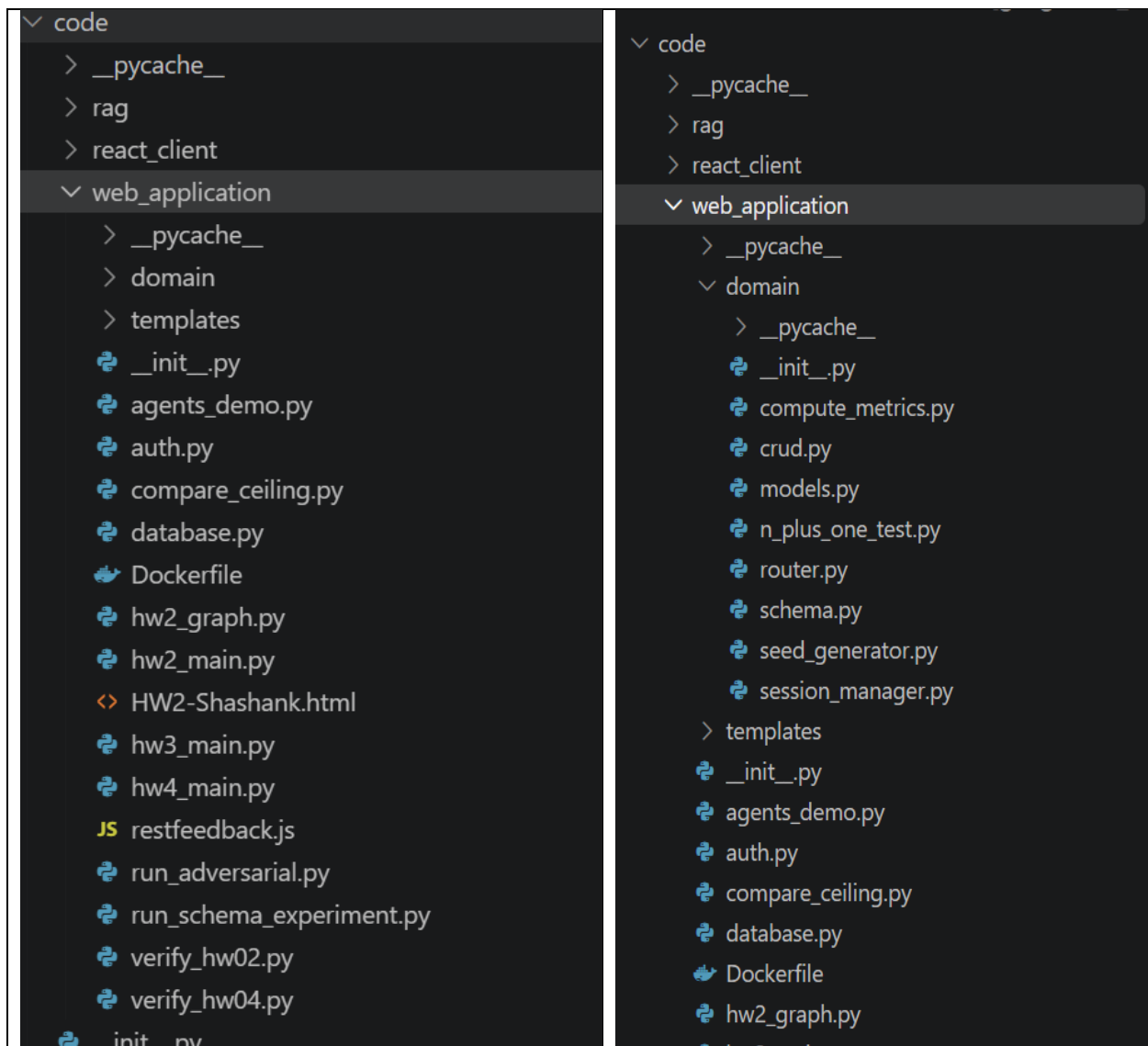
Local Windows PC, Python 3.13, VS Code Desktop, i7 CPU, 16GB RAM.

Model Used:

- Embeddings: sentence-transformers/all-MiniLM-L6-v2
- Ollama (qwen2.5:1.5b, qwen3:4b, llama3.2:1b)

Base Directory Structure:





HW4 Part 1: React Client

Summary:

The React bootstrapping flow in this project is standard and follows a clear structure:

index.html → **main.jsx** → **App.jsx** → **Components**

The index.html file contains the root <div> where the entire React application will be mounted. It loads **main.jsx** as a module. In **main.jsx**, React creates the application and renders the App component into the root element. From there, App.jsx defines the main browser routes and loads individual React components for the Home page, Login page, and the different CRUD pages.

The React client works together with a FastAPI backend running on port 8137. The backend handles all data-related logic and manages authentication using session cookies. These cookies are HTTP-only, meaning they cannot be accessed or modified by JavaScript in the browser. This adds an extra layer of security.

On the React side, the authentication status is checked through a custom hook called `useAuth`. This hook sends a request to the `/me` endpoint in the FastAPI backend. If the session cookie is valid, the backend confirms that the user is authenticated; otherwise, it returns an error. The frontend uses this response to know whether the user is logged in.

Although the React app allows navigation to the CRUD pages, the actual operations—creating, reading, updating, and deleting records—are restricted by the backend. Only authenticated users can perform these actions. If a user is not logged in, the backend simply blocks the request, ensuring that important operations remain secure even though the interface itself is accessible.

This structure keeps the frontend simple while relying on backend session validation to protect all CRUD operations.

```
de > react_client > <> index.html > html
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <title>DATA260 HW4 - Restaurant Inspection</title>
6    </head>
7    <body>
8      <div id="root"></div>
9      <script type="module" src="/src/main.jsx"></script>
10   </body>
11 </html>
```

```
main.jsx X
code > react_client > src > main.jsx
1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import App from "./App.jsx";
4
5  ReactDOM.createRoot(document.getElementById("root")).render(
6    <React.StrictMode>
7      <App />
8    </React.StrictMode>
9  );
```

Code : App.jsx (under react_client\src)

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Login from "../component/Login";
import Home from "../component/Home";
import CreateRecord from "../component/CreateRecord";
import UpdateRecord from "../component/UpdateRecord";
import DeleteRecord from "../component/DeleteRecord";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/create" element={<CreateRecord />} />
        <Route path="/update/:id" element={<UpdateRecord />} />
        <Route path="/delete/:id" element={<DeleteRecord />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Component Code Snippets and Workflows

I. Home Page (Home.jsx)

The Home component is displayed on the main route. It fetches and shows all inspection records from the backend. If the user is not logged in, the page can show a message such as “Login required” and hide any buttons that lead to creating, updating, or deleting records. This keeps the interface simple while still relying on the backend to enforce authentication.

```
import { useEffect, useState } from "react";
import api from "../api";
import useAuth from "../hooks/useAuth";
import { Link, useNavigate } from "react-router-dom";

export default function Home() {
  const authenticated = useAuth();
  const navigate = useNavigate();
  const [records, setRecords] = useState([]);

  useEffect(() => {
    if (authenticated === false) {
      navigate("/login");
    } else if (authenticated === true) {
      api.get("/inspection/")
        .then(res => setRecords(res.data))
        .catch(() => {});
    }
  }, [authenticated]);

  if (authenticated === null) return <p>Loading...</p>;
  .....
```

II. Login Page (Login.jsx)

The Login component provides a basic login form where the user enters an email and password. When the form is submitted, the component sends a request to the backend’s /login route. If the login is successful, a session cookie is created by the server, and the user is redirected back to the Home page. If the login fails, the component displays an alert so the user can retry.

```
import { useState } from "react";
import api from "../api";
import { useNavigate } from "react-router-dom";

export default function Login() {
```

```

const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const navigate = useNavigate();

const handleLogin = async (e) => {
  e.preventDefault();
  try {
    await api.post("/login", null, {
      params: { email, password }
    });
    navigate("/");
  } catch (err) {
    console.error("Login error:", err.response?.status, err.response?.data);
    alert("Invalid login or login failed. Please check your credentials and try again.");
  }
};

return (
  <div>
    <h2>Login</h2>
  </div>
);

```

.....

III. Create Record (CreateRecord.jsx)

This component appears on the /create route. It uses state variables to store the name and status of a new inspection record. When the user submits the form, the component sends a POST request to the backend to save the new record in the database. After the operation succeeds, the user is redirected to the main page showing all records.

```

import { useState } from "react";
import api from "../api";
import { useNavigate } from "react-router-dom";

export default function CreateRecord() {
  const [name, setName] = useState("");
  const [status, setStatus] = useState("");
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    await api.post("/inspection/", {
      name,
      status
    });
    navigate("/");
  };

  return (
    <div>
      <h2>Create Record</h2>
      <form>
        <input type="text" value={name} onChange={e => setName(e.target.value)} />
        <input type="text" value={status} onChange={e => setStatus(e.target.value)} />
        <button type="submit">Submit</button>
      </form>
    </div>
  );
}

```

```

    });
    navigate("/");
  };

  return (
    <div>
      <h2>Create Inspection Record</h2>

```

IV. Update Record (UpdateRecord.jsx)

The Update component loads when the user navigates to /update/:id. It reads the record ID from the route and fetches the existing information so it can pre-fill the form fields. The user can change the name or status, and when the form is submitted, the component sends a PUT request to update the record in the MySQL database. After the update is complete, the user is redirected back to the main list.

```

import { useState, useEffect } from "react";
import api from "../api";
import { useParams, useNavigate } from "react-router-dom";

export default function UpdateRecord() {
  const { id } = useParams();
  const navigate = useNavigate();

  const [name, setName] = useState("");
  const [status, setStatus] = useState("");

  useEffect(() => {
    api.get(`/inspection/${id}`)
      .then(res => {
        setName(res.data.name);
        setStatus(res.data.status);
      });
  }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();
    await api.put(`/inspection/${id}`, {
      name,
      status
    });
    navigate("/");

```

```
};

return (
  <div>
    <h2>Update Record</h2>
```

V. Delete Record (DeleteRecord.jsx)

The Delete component appears on `/delete/:id` and shows a confirmation prompt for deleting a specific record. When the user confirms, the component sends a DELETE request to the backend. If the removal is successful, the user is redirected back to the page that lists all inspection records.

```
import api from "../api";
import { useParams, useNavigate } from "react-router-dom";

export default function DeleteRecord() {
  const { id } = useParams();
  const navigate = useNavigate();

  const handleDelete = async () => {
    await api.delete(`/inspection/${id}`);
    navigate("/");
  };

  return (
    <div>
      <h2>Delete Record</h2>
      <p>Are you sure you want to delete this record?</p>
      <button onClick={handleDelete}>Delete</button>
    </div>
  );
}
```

Execution and log:

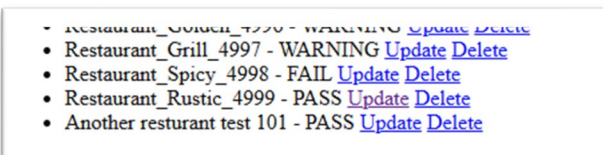
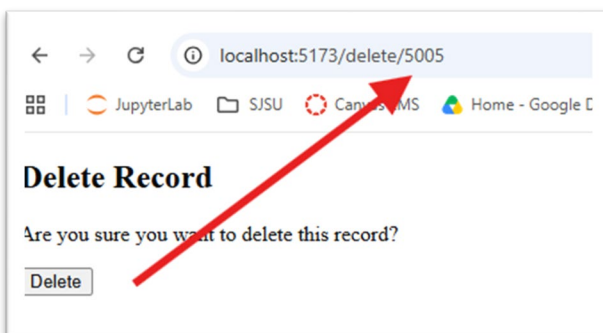
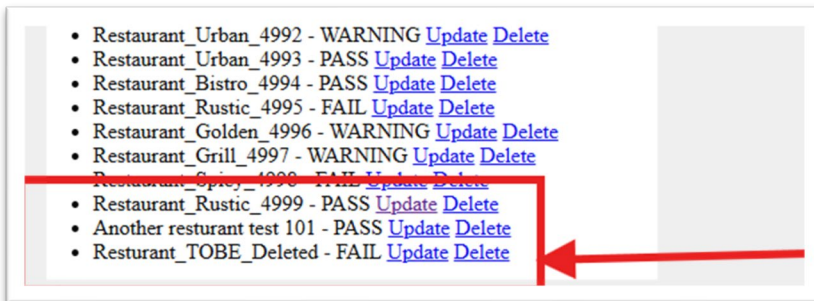
```
C:\SJSURepo\Data260\Data260-2837\code\react_client>npm run dev

> react_client@0.0.0 dev
> vite

VITE v5.4.21 ready in 1638 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Added/Updated/Deletions Restaurant: (here deletion steps)



Moreover, validated with “un-authorized” or logout session (after forcefully deleting the cookies). Once I deleted the “session cookies”, it directly redirecting to login page.

HW4 Part 2: MySQL Persistence and Server-Side Sessions

Introduction

This document describes the implementation of a simple CRUD web application using FastAPI, SQLAlchemy, and MySQL.

The application includes server-side session management, user authentication, and complete Create/Read/Update/Delete operations for the selected domain entity.

All database interactions use SQLAlchemy ORM, and session handling uses opaque tokens stored securely in the database.

System Overview

The backend application runs using Uvicorn.

Example startup command:

```
uvicorn hw4_main:app --port 8137
```

implemented the following major components:

- MySQL database named <PREFIX>_rel
- SQLAlchemy ORM models
- CRUD routes for the domain entity
- User login and session management
- Protected routes using dependency injection
- Postman verification for all API operations

Database Creation

The required database is created using the naming format <PREFIX>_rel.

In this project, the database used is:

- s2837_rel

Tables

A table for the primary domain entity was created.

My domain entity: **restaurant_inspection**

Fields:

- id
- name
- status

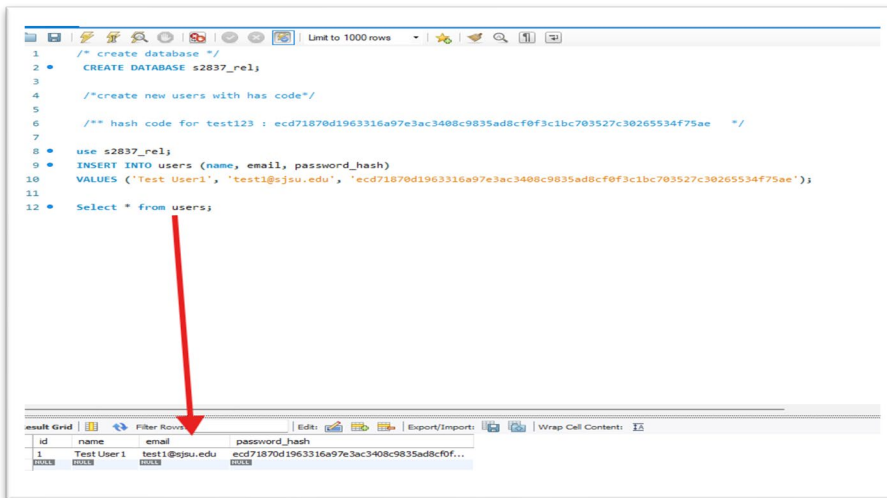
Authentication Tables

Two tables were created to support login and session handling.

users table

Fields:

- id
- name
- email (UNIQUE)
- password_hash

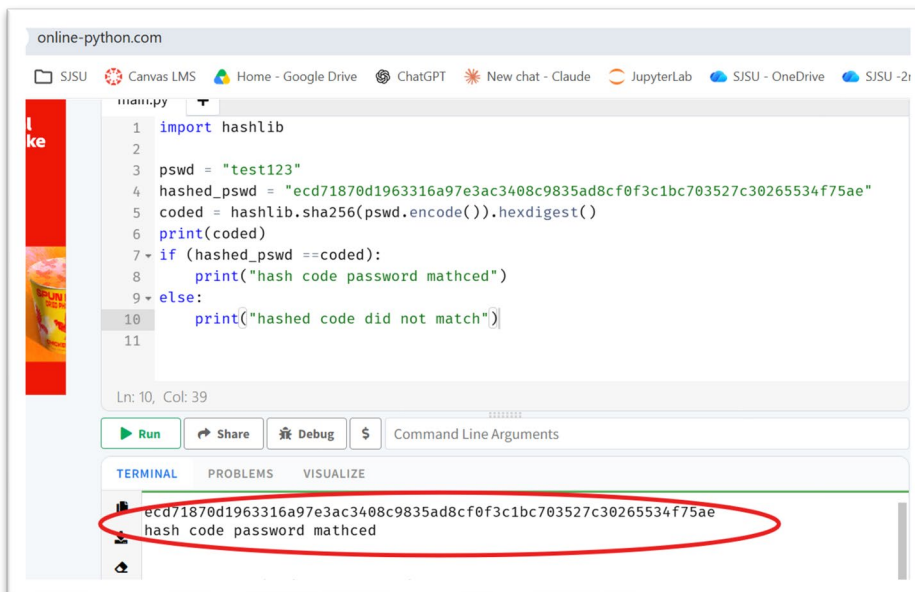


The screenshot shows a database client interface with a SQL editor at the top and a table viewer at the bottom. The SQL editor contains the following commands:

```
1 /* create database */
2 CREATE DATABASE s2837_rel;
3
4 /*create new users with has code*/
5
6 /** hash code for test123 : ecd71870d1963316a97e3ac3408c9835ad8cf0f3c1bc703527c30265534f75ae */
7
8 use s2837_rel;
9 INSERT INTO users (name, email, password_hash)
10 VALUES ('Test User1', 'test1@sjsu.edu', 'ecd71870d1963316a97e3ac3408c9835ad8cf0f3c1bc703527c30265534f75ae');
11
12 Select * from users;
```

A red arrow points from the SQL editor to the table viewer. The table viewer displays the following data:

id	name	email	password_hash
1	Test User1	test1@sjsu.edu	ecd71870d1963316a97e3ac3408c9835ad8cf0f3c1bc703527c30265534f75ae



The screenshot shows an online Python interpreter interface. The code in the editor is as follows:

```
1 import hashlib
2
3 pswd = "test123"
4 hashed_pswd = "ecd71870d1963316a97e3ac3408c9835ad8cf0f3c1bc703527c30265534f75ae"
5 coded = hashlib.sha256(pswd.encode()).hexdigest()
6 print(coded)
7 if (hashed_pswd == coded):
8     print("hash code password mathced")
9 else:
10     print("hashed code did not match")
11
```

The output in the terminal is:

```
ecd71870d1963316a97e3ac3408c9835ad8cf0f3c1bc703527c30265534f75ae
hash code password mathced
```

A red oval highlights the output in the terminal.

- A password hash for test123 was generated using Python's hashlib.
- A test user was inserted into the database.

sessions table

Fields:

- id (session token)
- user_id
- created_at
- expires_at

This table stores server-side session tokens.

The browser receives only an HTTP-only cookie containing an opaque session ID.

ORM and Database Connection

The backend uses SQLAlchemy ORM to define the models and handle database operations.

The assignment requires the database connection variable to be:

- db_session_basede26

This variable is used inside the dependency injection system.

Models.py (ORM Model Definitions) using sqlalchemy

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from datetime import datetime, timedelta
from database import Base

# MAIN DOMAIN ENTITY: restaurant inspection record
class RestaurantInspection(Base):
    __tablename__ = "restaurant_inspection"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255), nullable=False) # primary field
    status = Column(String(255), nullable=False) # secondary field

    # related rows will be used for N+1 (Part 3)
    related_items = relationship("InspectionRelated")

# RELATED TABLE FOR N+1 (200 rows)
class InspectionRelated(Base):
    __tablename__ = "inspection_related"

    id = Column(Integer, primary_key=True)
    inspection_id = Column(Integer, ForeignKey("restaurant_inspection.id"))
    details = Column(String(255))
```

```

# USERS TABLE (login)
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(255))
    email = Column(String(255), unique=True, index=True)
    password_hash = Column(String(255))

# SESSIONS TABLE (server-side tokens)
class SessionToken(Base):
    __tablename__ = "sessions"

    id = Column(String(255), primary_key=True)    # opaque token
    user_id = Column(Integer, ForeignKey("users.id"))
    created_at = Column(DateTime, default=datetime.utcnow)
    expires_at = Column(DateTime)

```

Dependency Injection Mechanisms

Database Dependency: Depends(get_db) manages the database session lifecycle:

- Opens a session per HTTP request
- Ensures proper closure after response
- Prevents unused or hanging database connections

```

database_url = f"mysql+pymysql://{username}:{password}@localhost:{port}/{database_name}"
#DATABASE_URL = "mysql+pymysql://root:YOUR_PASSWORD@localhost:3306/s2837_rel"

engine = create_engine(
    database_url,
    pool_pre_ping=True,
    echo=False
)

query_counter = {"count": 0}

# REQUIRED variable name EXACTLY as homework specifies
db_session_basede26 = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = db_session_basede26()
    try:
        yield db
    finally:
        db.close()

```

Authentication Dependency :Depends(require_login) ensures that protected routes are accessible only to authenticated users:

- Validates session token stored in the sessions table
- Returns HTTP 401 if the session is invalid or expired
- Allows access if the session is valid

```
# -----  
# Helper - Check login status using HTTP-only session cookie  
# -----  
def require_login(  
    request: Request,  
    db: Session = Depends(get_db),  
):  
    token = request.cookies.get("session_token")  
    if not token:  
        raise HTTPException(status_code=401, detail="Login required")  
  
    user_id = validate_session(db, token)  
    if not user_id:  
        raise HTTPException(status_code=401, detail="Session expired or invalid")  
  
    return user_id  # parent routes can use this  
#
```

CRUD Operations

The application implements full CRUD functionality for the domain entity:

1. Create (POST) – Add a new record
2. Read (GET) – View all records
3. Read (GET /id) – View a specific record
4. Update (PUT) – Modify an existing record
5. Delete (DELETE) – Remove a record

All CRUD operations use SQLAlchemy for database access.

```
from sqlalchemy.orm import Session  
from .models import RestaurantInspection  
  
def get_all(db: Session):  
    return db.query(RestaurantInspection).all()  
  
def get_one(db: Session, id: int):  
    return db.query(RestaurantInspection).filter(RestaurantInspection.id == id).first()
```

```

def create_record(db: Session, data):
    obj = RestaurantInspection(name=data.name, status=data.status)
    db.add(obj)
    db.commit()
    db.refresh(obj)
    return obj

def update_record(db: Session, id: int, data):
    obj = get_one(db, id)
    if obj:
        obj.name = data.name
        obj.status = data.status
        db.commit()
        db.refresh(obj)
    return obj

def delete_record(db: Session, id: int):
    obj = get_one(db, id)
    if obj:
        db.delete(obj)
        db.commit()
    return obj

```

This section includes the responses captured from Postman during testing.

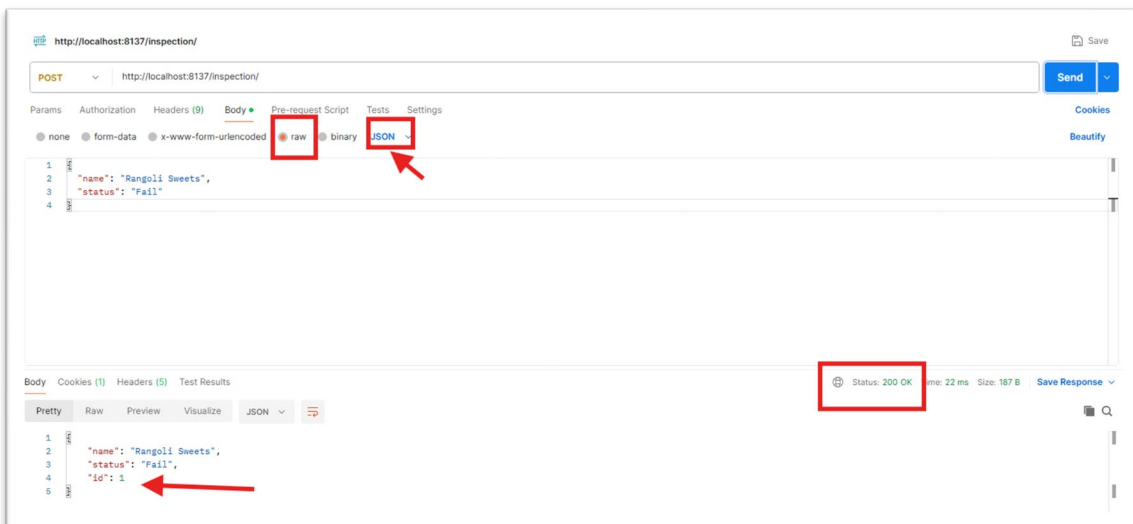
```

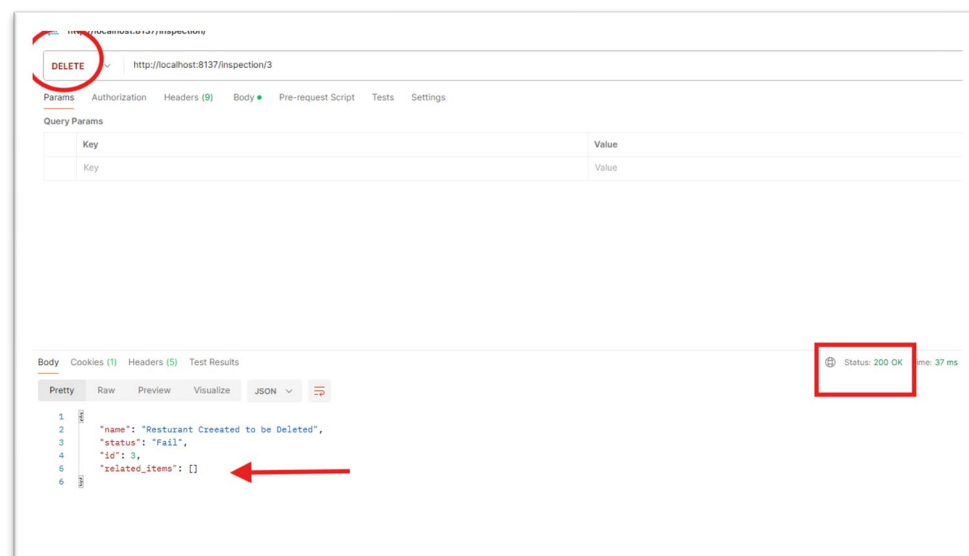
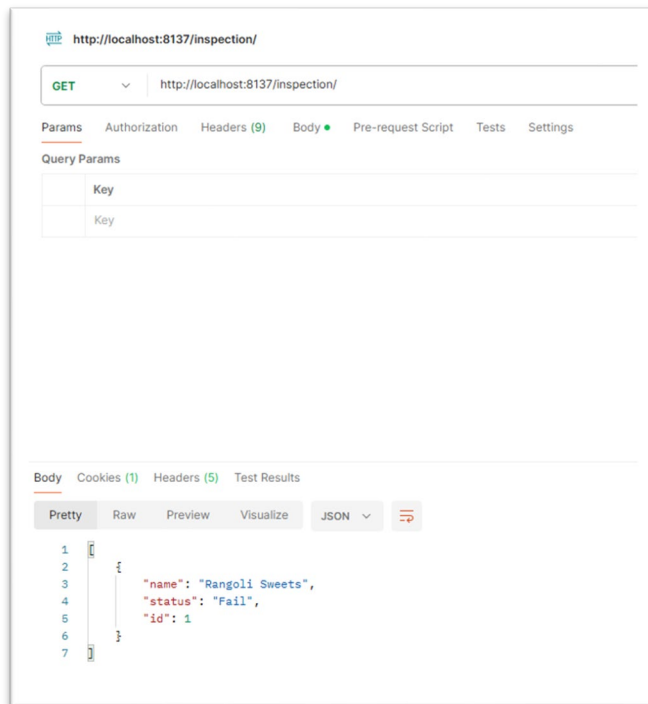
1 GET /
{
  "message": "DATA260 HW4 Backend Running",
  "port": 8137
}
2 POST /login
{
  "message": "Login successful"
}
3 GET /me
{
  "user_id": 1,
  "status": "authenticated"
}
4 POST /inspection/
{

```

```
"name": "Rangoli Sweets",
"status": "Fail",
"id": 1
}
5 GET /inspection/
[
{
  "id": 1,
  "name": "Rangoli Sweets and Snacks",
  "status": "Pass"
},
{
  "id": 2,
  "name": "Moonlight Garden Restaurant & Banquet Hall",
  "status": "Fail"
}
]
6 DELETE /inspection/3
{
  "name": "Resturant Creeated to be Deleted",
  "status": "Fail",
  "id": 3,
  "related_items": []
}
```

Postman Screenshot :





Summary

This section demonstrates the implementation of a complete CRUD application using FastAPI and MySQL. It includes user authentication, session management, REST API endpoints, and database persistence using SQLAlchemy.

The application follows all assignment requirements including database structure, naming conventions, session handling, and Postman verification.

HW4 Part 3: N+1 Measurement and Query Tuning

This section explains how the N+1 query problem was measured, how it was fixed, and how performance changed across different page sizes. It also includes a simple explanation of the database index added and how it improves query performance.

Seeding the Database

Using the seed generator script (seed_generator.py) with SEED = 2837, the database was populated with:

- 5,000 main records in the restaurant_inspection table
- 200 related records in a test table named inspection_related

Each related row was linked back to one of the main inspection records. This setup provides enough data to test how the list endpoint behaves under load.

```
import random
from sqlalchemy.orm import Session
from .models import RestaurantInspection, InspectionRelated
from database import db_session_based26, Base, engine

SEED = 2837
random.seed(SEED)

def run_seed():
    Base.metadata.create_all(bind=engine)
    db: Session = db_session_based26()

    word_choices = ["Spicy", "Golden", "Urban", "Rustic", "Bar", "Garden", "Bistro", "Cafe",
                    "Deli", "Grill"]
    # 1. Seed 5000 primary records
    inspections = []
    for i in range(5000):
        random_words = random.choice(word_choices)
        name = f"Restaurant_{random_words}_{i}"
        status = random.choice(["PASS", "FAIL", "WARNING"])
        obj = RestaurantInspection(name=name, status=status)
        db.add(obj)
        inspections.append(obj)

    db.commit()

    # 2. Seed 200 related rows
```

```

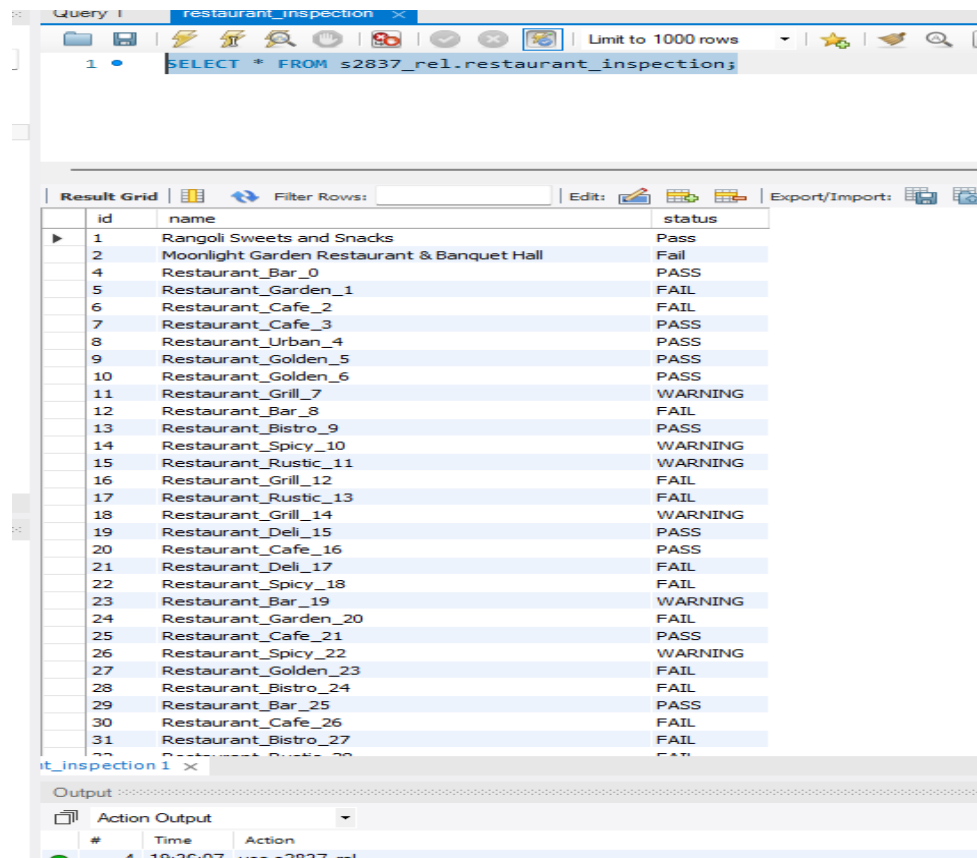
for i in range(200):
    parent = inspections[random.randint(0, 4999)]
    related_text = f"Related details row {i}"
    related = InspectionRelated(inspection_id=parent.id, details=related_text)
    db.add(related)

db.commit()
db.close()
print("Seeding completed.")

if __name__ == "__main__":
    run_seed()

```

Here is the validation from the database by executing the query:



The screenshot shows a database query tool interface. At the top, a query editor displays the SQL query: `SELECT * FROM s2837_rel.restaurant_inspection;`. Below the query editor, a 'Result Grid' shows the results of the query. The grid has three columns: 'id', 'name', and 'status'. The results are listed in a table format with alternating blue and white rows. At the bottom of the screenshot, an 'Output' section shows a log entry: '4 19:36:07 use s2837 rel'.

id	name	status
1	Rangoli Sweets and Snacks	Pass
2	Moonlight Garden Restaurant & Banquet Hall	Fail
4	Restaurant_Bar_0	PASS
5	Restaurant_Garden_1	FAIL
6	Restaurant_Cafe_2	FAIL
7	Restaurant_Cafe_3	PASS
8	Restaurant_Urban_4	PASS
9	Restaurant_Golden_5	PASS
10	Restaurant_Golden_6	PASS
11	Restaurant_Grill_7	WARNING
12	Restaurant_Bar_8	FAIL
13	Restaurant_Bistro_9	PASS
14	Restaurant_Spicy_10	WARNING
15	Restaurant_Rustic_11	WARNING
16	Restaurant_Grill_12	FAIL
17	Restaurant_Rustic_13	FAIL
18	Restaurant_Grill_14	WARNING
19	Restaurant_Deli_15	PASS
20	Restaurant_Cafe_16	PASS
21	Restaurant_Deli_17	FAIL
22	Restaurant_Spicy_18	FAIL
23	Restaurant_Bar_19	WARNING
24	Restaurant_Garden_20	FAIL
25	Restaurant_Cafe_21	PASS
26	Restaurant_Spicy_22	WARNING
27	Restaurant_Golden_23	FAIL
28	Restaurant_Bistro_24	FAIL
29	Restaurant_Bar_25	PASS
30	Restaurant_Cafe_26	FAIL
31	Restaurant_Bistro_27	FAIL
32	Restaurant_Spicy_28	FAIL

Related Data Included in Responses

The list endpoint returns restaurant inspection records and also includes the related inspection information. This ensures every record triggers at least one lookup in the related table when using the naive approach.

Naive Endpoint and the N+1 Problem

A naive implementation was built intentionally for testing. It does the following:

- Runs one main query to fetch the list of inspection records
- For each record returned, it performs an additional query to load the related data

This means the total query count becomes:

1 main query + N related queries = N+1 total queries.

This causes extra database round trips and a slowdown when page sizes grow.

Benchmark Testing

Tests were run with three page sizes: 10, 50, and 200.

For each page size:

- 30 requests were executed
- Total number of SQL queries per request was recorded
- Latency numbers were measured (p50, p95, p99)

All raw results were saved in:

reports/hw04/raw/

version,page_size,run,latency_ms,sql_queries
naive,10,0,20.180940628051758,11
naive,10,1,17.45772361755371,11
naive,10,2,10.081291198730469,11
naive,10,3,15.735387802124023,11

Fixed Version (Using Join or Eager Loading)

The fixed implementation removes the N+1 problem. Instead of issuing one query per record, the **system loads all required data in a single SQL query using a join (or eager loading)**.

This reduces the SQL query count to exactly 1 query, no matter what page size is used.

Benchmark Summary

Below is a simplified version of the recorded metrics.

Page Size 10

- Naive: 11 queries, slower latency
- Fixed: 1 query, about 2.5× faster

Page Size 50

- Naive: 51 queries, slower latency
- Fixed: 1 query, about 5.6× faster

Page Size 200

- Naive: 201 queries, significantly slower
- Fixed: 1 query, about 8.4× faster

Explanation:

The naive version performs one query per row. As page size increases, the number of queries increases directly with it:

- page_size=10 , It required 11 queries
- page_size=50 It required 51 queries
- page_size=200 It required 201 queries

Since each query adds its own round-trip time, latency grows almost linearly.

The fixed version (using join) always uses **one query**, so latency changes only slightly as page size grows. The slight increase is due to returning more rows at once, not due to more queries.

As a result, the difference between naive and fixed versions becomes larger when the page size increases.

The pattern is exactly right: naive scales query count and latency with page size (11→201 queries, ~15ms→~150ms), while fixed stays flat (1 query, low latency) regardless of page size.

```
C:\SJSURRepo\Data260\Data260-2837\code\web_application\domain>python compute_metrics.py
page_size=10 version=naive n=30 sql=11 p50=14.74ms p95=19.88ms p99=20.50ms
page_size=10 version=fixed n=30 sql=1 p50=5.78ms p95=15.46ms p99=16.76ms
page_size=50 version=naive n=30 sql=51 p50=37.83ms p95=47.91ms p99=48.43ms
page_size=50 version=fixed n=30 sql=1 p50=6.75ms p95=16.69ms p99=18.25ms
page_size=200 version=naive n=30 sql=201 p50=132.73ms p95=149.07ms p99=151.46ms
page_size=200 version=fixed n=30 sql=1 p50=15.74ms p95=36.24ms p99=50.80ms

Written to ../../reports/hw04/METRICS.md

C:\SJSURRepo\Data260\Data260-2837\code\web_application\domain>
```

Database Index Optimization

To improve filtering performance, an index was added on the status column in the restaurant_inspection table.

Query Tested

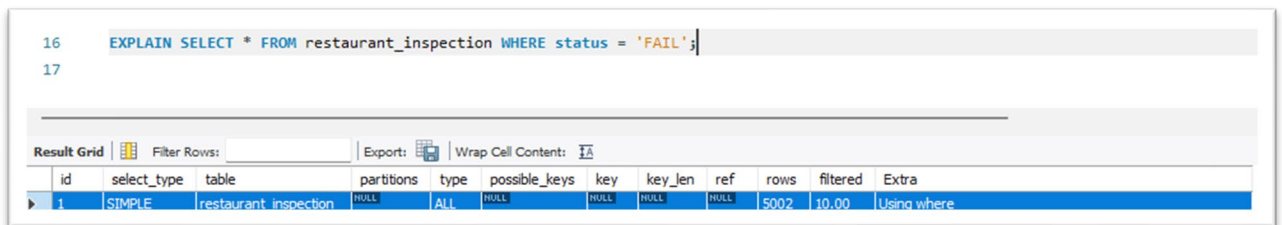
EXPLAIN SELECT * FROM restaurant_inspection WHERE status = 'FAIL';

Before Indexing

- MySQL performed a full table scan
- Every row had to be checked
- About 5,002 rows were examined
- No index was used

"before" result:

- type: ALL — full table scan, MySQL checks every row one by one
- key: NULL — no index used at all
- rows: 5002 — has to examine essentially the entire table
- Extra: Using where — it's filtering after reading everything, rather than jumping straight to matches



```
16 EXPLAIN SELECT * FROM restaurant_inspection WHERE status = 'FAIL';
17
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	restaurant_inspection	NULL	ALL	NULL	NULL	NULL	NULL	5002	10.00	Using where

Index Added (on status)

CREATE INDEX idx_restaurant_inspection_status ON restaurant_inspection(status);

After Indexing

- MySQL used the new index
- Only the rows matching the filter were scanned (about one-third of the table)
- Query became much more efficient

```

18  /* creating index on the status column of this table */
19  • CREATE INDEX idx_restaurant_inspection_status ON restaurant_inspection (status);
20  • SHOW INDEX FROM restaurant_inspection;

```

result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Su
restaurant_inspection	0	PRIMARY	1	id	A	5002	NULL
restaurant_inspection	1	ix_restaurant_inspection_id	1	id	A	5002	NULL
restaurant_inspection	1	idx_restaurant_inspection_status	1	status	A	3	NULL

```

23  • EXPLAIN SELECT * FROM restaurant_inspection WHERE status = 'FAIL';
24

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	restaurant_inspection	NULL	ref	idx_restaurant_inspection_status	idx_restaurant_inspection_status	1022	const	1660	100	NULL

Explanation: Without an index, the database must check every row even if only a small portion matches the condition. With the index, the database jumps directly to matching rows, reducing work and improving performance.

The foreign key column (inspection_id) in the related table already had an index created automatically by the database system. This prevented additional indexing issues.

The main slowdown in the naive version was not indexing—it was the high number of queries caused by the N+1 pattern.

HW4 Part 4: Build a Grounded RAG Question-Answering System

Introduction

This assignment/section describes a small Retrieval-Augmented Generation (RAG) system designed to answer questions using only information stored in a collection of domain-specific inspection documents. The goal is to show how retrieval quality and context formatting influence answer accuracy, grounding, and refusal behavior. The system evaluates three different configurations and includes tests for both answerable and unanswerable questions.

Pipeline Overview

- **Corpus and Chunking**

The system uses six inspection-related text documents. Each document is divided into smaller parts called chunks. The chunk size is set to 500 characters, with an overlap of 50 characters to avoid splitting important information. After processing all documents, 99 total chunks are created. Each chunk stores the text, its original file name, and a unique identifier.

- **Model and Embeddings**

All chunks are converted into embeddings using a sentence-transformer model. These embeddings are stored in a FAISS index. The index allows fast retrieval of the most relevant chunks for any given question based on similarity scores.

System Configurations

- **No-RAG Configuration**

In this setup, the language model receives only the user's question. No document context is provided. This acts as a baseline and shows how the model behaves without grounding.

- **Basic RAG Configuration**

The model receives the question along with the top three retrieved chunks. These chunks are passed directly into the prompt without any reordering or filtering. This setup demonstrates how raw retrieval affects the answer.

- **Context-Engineered RAG Configuration**

This configuration improves context quality by removing duplicate chunks, labeling each chunk with source information, and adding strict grounding rules. The model is instructed to answer only using the provided evidence. If the context does not contain the required information, the model must respond with exactly:

"I cannot answer this question from the provided documents."

Retrieved Chunks Summary

```
# -----  
# 7. Six Test Questions Evaluation  
# -----  
questions = [  
  {"id": "Q1", "type": "Single Chunk", "q": "What are the required daily cleaning tasks for  
restaurant food preparation areas?"},
```

```
{
  "id": "Q2", "type": "Two Chunks", "q": "Why did the display case cooling failure at Rangoli Sweets violate county temperature standards?"},
  "id": "Q3", "type": "Multi-Doc", "q": "What signs of pest activity have been identified in restaurant inspections?"},
  "id": "Q4", "type": "Ambiguous", "q": "Under what conditions should a restaurant be temporarily closed?"},
  "id": "Q5", "type": "Not in Docs", "q": "What are the minimum wage and overtime pay requirements for kitchen staff?"},
  "id": "Q6", "type": "Unrelated", "q": "How do I assess the value of a single family home in Santa Clara County?"}
]
```

Question 1

The top chunks came from documents related to kitchen sanitation and food safety. They focused on cleaning procedures and daily maintenance tasks.

Question 2

All retrieved chunks originated from the same document discussing cooling violations. These chunks contained details about observed temperatures and county standards.

Question 3

Chunks were retrieved from multiple documents referencing different pest signs. This shows the system's ability to combine evidence from various sources.

Question 4

The retrieved chunks included guidance about unsafe conditions that may require temporary restaurant closure.

Question 5

This question asked about wage requirements, which are not present in any of the documents. The retrieved chunks were unrelated, showing high distance scores.

Question 6

This question dealt with home valuation, which is outside the scope of restaurant inspection documents. Retrieval again returned unrelated chunks.

Context Size Evaluation

- $k = 1$
With one chunk, the retrieval was very focused but missed some important details. The context did not include all required cleaning steps.
 - $k = 3$
This setting captured the complete information needed to answer the question. The context stayed relevant and avoided unnecessary text.
-
- $k = 5$
Retrieving five chunks introduced unrelated details from other documents. This increased token usage without adding helpful information.

Overall, a context size of three chunks provided the best balance of completeness and relevance.

Evaluation Summary

For Questions 1 through 4, the system retrieved relevant chunks and produced correct and grounded answers. The responses were supported by the evidence supplied.

For Question 5, the system correctly refused to answer when using the context-engineered configuration because the required information was not present in any document.

For Question 6, the system generally refused, although at times it attempted to provide outside information. This highlights the importance of strong grounding rules and possibly additional filtering for poorly matched chunks.

Context Engineering and Final Analysis

The overall results show that retrieval quality, context organization, and prompt instructions all play important roles in RAG performance. When relevant chunks are present, the model can produce accurate and reliable answers. Cleaning and labeling the context help the model focus on the information that truly matters. Grounding rules prevent the model from using external knowledge, which is essential for maintaining faithfulness to the document set.

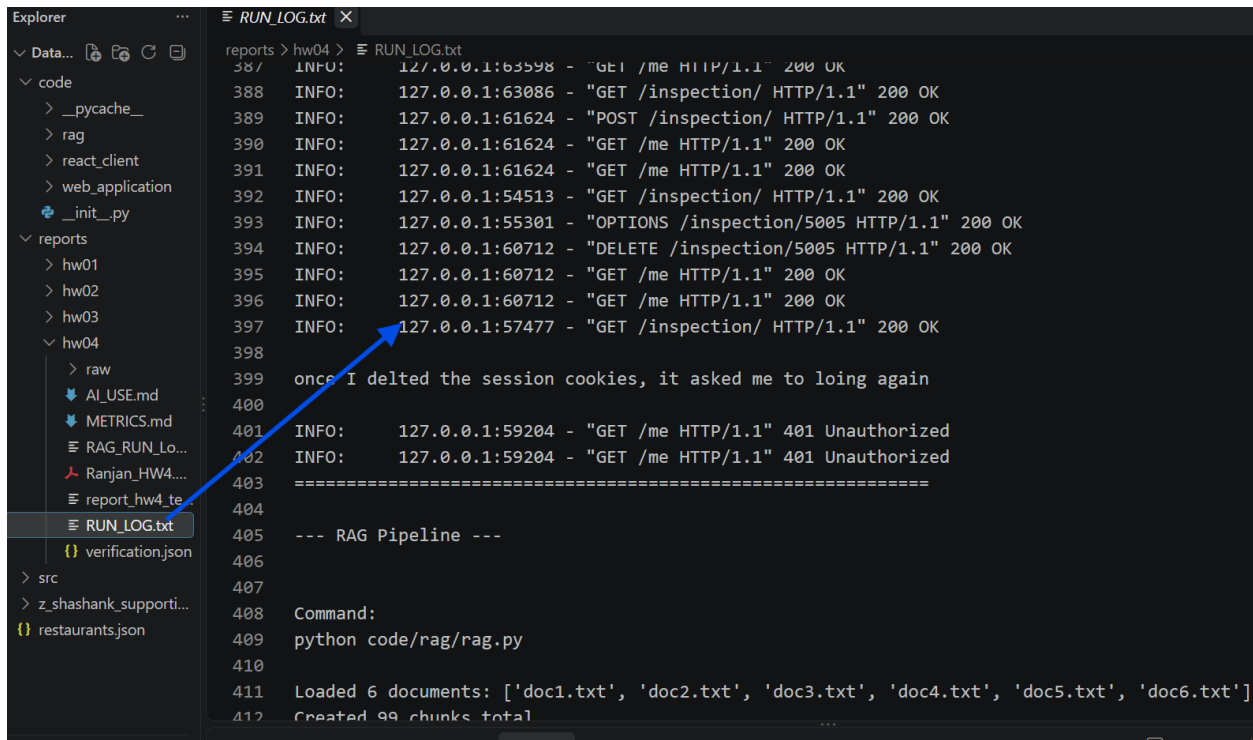
Increasing the amount of context does not always improve answers. A smaller and well-selected set of chunks often performs better than a larger set with irrelevant details. The tests with unanswerable questions further demonstrate that strict instructions are necessary for consistent refusal behavior.

In summary, answer quality depends on three factors working together: correct retrieval, well-structured context, and clear grounding rules. When these are combined effectively, the system can produce accurate, grounded, and dependable responses.

Reporting Requirements

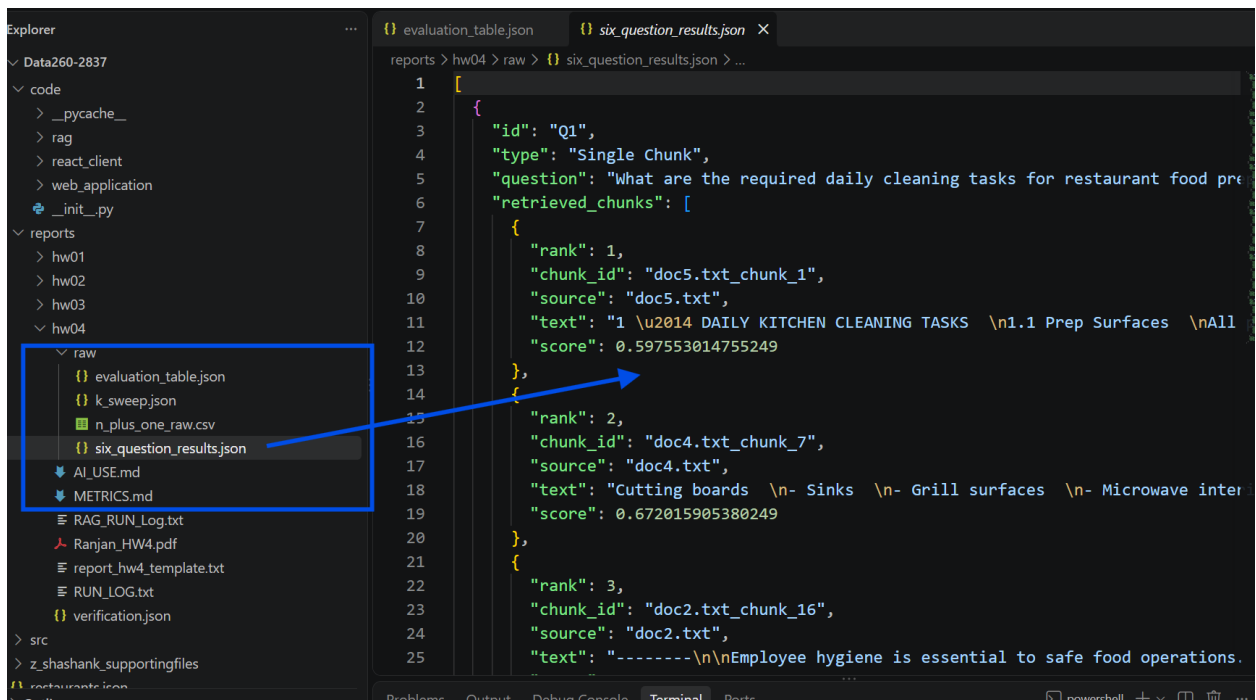
1. RUN_LOG.txt

Stored reports/hw04/RUN_LOG.txt shows details from command prompt



```
reports > hw04 > RUN_LOG.txt
387 INFO: 127.0.0.1:63598 - "GET /me HTTP/1.1" 200 OK
388 INFO: 127.0.0.1:63086 - "GET /inspection/ HTTP/1.1" 200 OK
389 INFO: 127.0.0.1:61624 - "POST /inspection/ HTTP/1.1" 200 OK
390 INFO: 127.0.0.1:61624 - "GET /me HTTP/1.1" 200 OK
391 INFO: 127.0.0.1:61624 - "GET /me HTTP/1.1" 200 OK
392 INFO: 127.0.0.1:54513 - "GET /inspection/ HTTP/1.1" 200 OK
393 INFO: 127.0.0.1:55301 - "OPTIONS /inspection/5005 HTTP/1.1" 200 OK
394 INFO: 127.0.0.1:60712 - "DELETE /inspection/5005 HTTP/1.1" 200 OK
395 INFO: 127.0.0.1:60712 - "GET /me HTTP/1.1" 200 OK
396 INFO: 127.0.0.1:60712 - "GET /me HTTP/1.1" 200 OK
397 INFO: 127.0.0.1:57477 - "GET /inspection/ HTTP/1.1" 200 OK
398
399 once I delted the session cookies, it asked me to loing again
400
401 INFO: 127.0.0.1:59204 - "GET /me HTTP/1.1" 401 Unauthorized
402 INFO: 127.0.0.1:59204 - "GET /me HTTP/1.1" 401 Unauthorized
403 =====
404
405 --- RAG Pipeline ---
406
407
408 Command:
409 python code/rag/rag.py
410
411 Loaded 6 documents: ['doc1.txt', 'doc2.txt', 'doc3.txt', 'doc4.txt', 'doc5.txt', 'doc6.txt']
412 Created 99 chunks total
```

2. Raw/ (details are under hw04/raw)



3. Metrics.md

1	Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)	
2	---	---	---	---	---	---	
3	10	naive	11	14.74	19.88	20.50	
4	10	fixed	1	5.78	15.46	16.76	
5	50	naive	51	37.83	47.91	48.43	
6	50	fixed	1	6.75	16.69	18.25	
7	200	naive	201	132.73	149.07	151.46	
8	200	fixed	1	15.74	36.24	50.80	
9							

4. AI_USE

AI_USE.md - HW4

1. What I used an AI assistant for

I used an AI assistant in a limited way to clarify certain framework conventions and to compare different approaches while planning my implementation. Specifically, I consulted it for:

examples of common FastAPI + SQLAlchemy project structures
general React component patterns
reviewing high-level RAG workflow options
checking typical ways to organize benchmarking or packaging scripts

All actual code, project files, and logic were written and implemented by me.

2. One AI-produced output that was wrong

The AI assistant provided an outdated FastAPI routing example that didn't match how my project was structured.

3. How I detected the problem

While testing the backend, the routing example caused unexpected behavior and didn't align with the patterns shown in the current FastAPI documentation. After reviewing the latest FastAPI and SQLAlchemy guidelines, I confirmed that the example was based on older conventions.

4. What I changed and why it works now

I rewrote the routing logic using the correct FastAPI pattern and verified it against the current documentation.

With the updated approach, the endpoints load properly and follow the expected request/response flow for my project.

Verification.json (under report\hw04)

--final verification

C:\SJSURepo\Data260\Data260-2837\code\web_application>python verify_hw04.py

verification.json created successfully.

```
{
  "homework": "HW4",
  "sid4": 2837,
  "seed": 2837,
  "verify_seed": 262837,
  "port": 8137,
  "model": "sentence-transformers/all-MiniLM-L6-v2 + local Ollama models",
  "checks": [
    {
```

```
    "check": "backend_running",  
    "status": "PASS"  
  },  
  {  
    "check": "login",  
    "status": "PASS"  
  },  
  {  
    "check": "me_authenticated",  
    "status": "PASS"  
  },  
  {  
    "check": "inspection_list",  
    "status": "PASS"  
  }  
]  
}
```